

Adaptive Camera Cleaning System

How it works, what I built, and how the vision model is integrated with the camera and the microcontroller. UIUC ECE 445 senior design, Spring 2025. Team of three. Repo: github.com/dbhan08/Weather-Resistant-Camera

1. What the system does

A weather-resistant enclosure keeps a camera lens usable in rain and frost. An STM32F4 microcontroller watches the lens with a small camera, decides when the lens is wet, and runs a servo-driven wiper. It also reads its on-die temperature sensor and switches a heating pad through a MOSFET when the temperature drops below a threshold. Both actions are automatic. No human input is needed once it is powered.

The project was split into seven subsystems: heater, temperature sensor, microcontroller, wiper, power, computer, and camera. The camera and computer subsystems are the vision path, which is the part I owned.

2. What I was responsible for

- **Camera subsystem.** Researched and selected the camera module (DFRobot FIT0701 for the bench demo), got a live feed working with OpenCV, and later wrote the DCMI capture path for the on-chip design.
- **Vision algorithm.** Wrote the raindrop detector: first the OpenCV HSV and contour heuristic used at the demo, then the CNN pipeline (pretrained port, fine-tune, distillation, int8 export) and the TensorFlow Lite Micro inference code that runs on the STM32.
- **Camera-to-MCU integration.** Defined the 5-byte serial protocol (START / STOPP) between the PC and the STM32, wrote the Python side, and then replaced it with an on-MCU loop so the PC is not in the loop.
- **Documentation.** Wrote the Introduction, Problem Statement and Proposed Solution in the proposal and design document, and the Requirements and Verification section of the final paper, including all verification data.
- **Shared hardware work.** PCB layout iterations in KiCad with the team, machine-shop enclosure meetings, soldering and bring-up of the first PCB.

3. Hardware and firmware overview

Block	Part / peripheral	What it does
MCU	STM32F4 (Nucleo dev board for the demo; custom PCB designed)	Runs the control loop, PWM, ADC, UART, DCMI
Wiper	Hobby servo on TIM2 CH1 PWM	Sweeps 60 to 115 degrees and back in 3-degree steps, 100 ms per step
Heater	Heating pad through a MOSFET on PA5, logic-level shifter, indicator LED	On when temperature is below threshold
Sensor	STM32 internal temperature sensor on ADC1	Factory-calibrated at 30 C and 110 C, minus a 9 C offset found in testing
Camera	OV7670 on DCMI + DMA (on-chip path); USB camera + PC (demo path)	160x120 RGB565 frames
Power	5 V and 3.3 V regulators	Servo and heater on 5 V, MCU on 3.3 V

Link	USART2, 115200 baud	Status log; START / STOPP commands in the demo path
------	---------------------	---

The servo angle is converted to a PWM compare value with a linear map from 0 to 180 degrees onto 210 to 1050 timer ticks:

```
void Set_Servo_Angle(TIM_HandleTypeDef *htim, uint32_t ch, uint8_t angle) {
    uint32_t pulse = 210 + (angle * (1050 - 210) / 180);
    __HAL_TIM_SET_COMPARE(htim, ch, pulse);
}
```

The heater trigger uses only the thermistor. Vision never drives the heater. Ice looks like many things on a lens; temperature does not. The wiper is also locked out when the temperature is at or below 0 C so the servo never fights ice.

4. Vision, version 1: the OpenCV heuristic used at the demo

The first plan was a CNN on the PC. It lagged the live feed, so for the demo I wrote a classical OpenCV pipeline in *final_code.py*. Every 4 seconds:

- 1 Grab a frame and flip it vertically, because the camera is mounted upside down in the enclosure.
- 2 Convert to HSV. Keep pixels with saturation below 40 (water on glass looks grey) AND value above 100 (rejects shadows).
- 3 Morphological close with a 15x15 ellipse to fill each drop into a solid blob.
- 4 Find external contours and sort by area. Skip the largest (the lens ring). Enclose the next two in minimum circles and fill them black, so a drop's bright interior counts as covered, not just its edge.
- 5 Compute the percentage of black pixels. Over 5 % sends the 5-byte string START to the STM32 over serial.

```
_, s_mask = cv2.threshold(s, 40, 255, cv2.THRESH_BINARY_INV)
_, v_mask = cv2.threshold(v, 100, 255, cv2.THRESH_BINARY)
closed = cv2.morphologyEx(cv2.bitwise_and(s_mask, v_mask), cv2.MORPH_CLOSE, kernel)
...
if black_percent > 5:
    ser.write(b"START") # STM32 sweeps the wiper
```

On the STM32, the main loop blocks up to 1 s on a 5-byte UART receive. START runs the sweep. STOPP is a no-op kept for protocol symmetry.

What went wrong

The heuristic false-fired on glare and surface texture. With the TA we found the root cause: the lens sits millimetres from the surface where drops form, so the camera is effectively zoomed in and small artifacts look like drops. I reworked the detector to key on the dark drop edges, then enclose and fill them to estimate true coverage, and ran 100 trials on varied test images to check it. That version is what ran at the final demo.

5. Vision, version 2: a CNN on the microcontroller

The CNN path removes the PC from the loop. The MCU captures the frame, classifies it, and triggers the wiper itself. The design is a **tile classifier**: the frame is cut into a fixed grid of tiles, each tile is labelled raindrop or clean, and the fraction of raindrop tiles is the coverage number. This is the coverage-metric idea from automotive lens-soiling work (Valeo SoilingNet / TiledSoilingNet) and it maps directly onto the 5 % rule from the heuristic.

5.1 Where the model comes from

No public model exists for water on a lens, so I started from the closest published one: the AlexNet-30² raindrop classifier from Guo, Akcay, Adey and Breckon (ICIP 2018), which reports 0.95 accuracy on 30x30 patches from a windshield camera. Its weights are a TensorFlow 1 / TFLearn checkpoint, which no current toolchain can run or convert. TF2 can still read TF1 checkpoint variables, so *port_breckon.py* rebuilds the exact graph in Keras and copies every tensor by name, asserting shapes:

```
_NAME_MAP = {"conv1": "Conv2D", "conv2": "Conv2D_1", ..., "fc3": "FullyConnected_2"}
reader = tf.train.load_checkpoint(tfl_path)
w, b = reader.get_tensor(f"{scope}/W"), reader.get_tensor(f"{scope}/b")
layer.set_weights([w, b])
```

5.2 Fine-tuning to our lens

Breckon's camera looked through a windshield. Ours is millimetres from the drops, so drops are larger and blurrier. *data_windshield.py* labels 30x30 patches from the RaindropsOnWindshield dataset (8,190 frames with pixel masks, CC BY 4.0). A patch is positive if the mask covers at least 30 % of it, and negative only if the patch and a one-patch margin are mask-free, so borderline drops never leak into negatives. Splits are by video sequence so no clip appears on both sides. *finetune.py* then does the standard two stages: dense head only at lr 1e-3, then all layers at 1e-5 with early stopping, with flips and 45-degree rotations as in the paper.

5.3 Why it had to be distilled

After three pooling layers a 30x30 input collapses to 1x1x256, followed by 256x4096 and 4096x4096 dense layers. That is about 18 million parameters: 75 MB in fp32, 19 MB even in int8. The STM32F4 has 512 KB of flash and 192 KB of SRAM. The model also uses local response normalization, which TensorFlow Lite Micro does not implement. So the fine-tuned network is the **teacher** on the PC and a small **student** runs on the chip.

The student (*distill_student.py*) takes a 48x48 tile, uses a stem convolution and five depthwise-separable blocks, global average pooling and two logits. About 20 k parameters, ReLU only, and just the ops TFLM has: CONV_2D, DEPTHWISE_CONV_2D, MEAN, FULLY_CONNECTED. It is trained with knowledge distillation (Hinton et al. 2015): the loss mixes KL divergence to the teacher's softened probabilities at temperature 4 with cross-entropy on the hard label. The soft labels carry the teacher's uncertainty on glare and texture, which is exactly the failure mode of the heuristic.

```
loss = alpha * KL(teacher_soft_T, student_soft_T) * T**2 + (1 - alpha) * CE(y, student_logits)
# T = 4, alpha = 0.7
```

5.4 Quantization and export

to_tflite.py converts the student to full-integer int8: int8 weights, int8 activations, int8 input and output. The converter needs a representative dataset, a few hundred real tiles it pushes through the fp32 graph to record each tensor's range. I use validation patches, not training patches, so the ranges are not tuned on the images

the weights were fitted to. The script then emits the model as a C array (*raindrop_student_int8.cc*) and a report with the byte size, op list, input scale and zero point, and int8-versus-fp32 argmax agreement.

6. Integration with the camera and the STM32

Three firmware files under *CNN/pretrained/tflm/* replace the PC. Memory budget: 38 KB RGB565 DMA frame, 58 KB RGB888 frame, 48 KB TFLM tensor arena, about 25 KB of model in flash.

6.1 Camera capture: camera_dcml.c

The OV7670 is configured over SCCB (I2C, address 0x42) for QQVGA 160x120 RGB565: reset, QVGA RGB mode, RGB565 output, enable the down-scaler and set the divider to /4 in both axes, auto gain / white balance / exposure on. Its pixel clock, HREF and VSYNC lines go to the DCMI peripheral, which DMA's one frame into SRAM in snapshot mode. The capture call blocks on the frame-complete callback with a 500 ms timeout, then converts RGB565 to RGB888 by replicating the high bits into the low bits so full white maps to 255, not 248.

```
HAL_DCMI_Start_DMA(&hdcmi, DCMI_MODE_SNAPSHOT, (uint32_t)s_frame565, (CAM_W*CAM_H*2)/4);
while (!s_frame_done) { if (HAL_GetTick() - t0 > 500) return 0; }
rgb888[3*i+0] = (r5 << 3) | (r5 >> 2); // and likewise g6, b5
```

6.2 Inference: raindrop_inference.cc

At boot, *RaindropInit* loads the model from the flash array, registers only the six ops the student uses in a MicroMutableOpResolver, allocates tensors in the 48 KB arena, and checks the input is int8 48x48x3 with a 2-way output. Per frame, *RaindropCoverage* walks the 3x2 grid of 48 px tiles. Each tile is copied out of the frame row by row and quantized with the scale and zero point the converter stored in the model. The student was trained on pixel/255, so the mapping is:

```
q = round((pixel / 255) / input_scale) + input_zero_point, clamped to [-128, 127]
```

After *Invoke*, the tile is raindrop if the second int8 logit is larger than the first. Argmax on quantized logits is order-preserving, so no dequantization is needed. The function returns raindrop tiles divided by total tiles and, optionally, the per-tile grid. The Cortex-M DWT cycle counter times the whole pass; *RaindropLastInferenceUs* exposes it for logging.

6.3 The control loop: main_raindrop.c

This is the original *main.c* with the UART trigger replaced by the CNN. Once per second:

- 1 Read the on-die temperature through ADC1 with the factory TS_CAL1 / TS_CAL2 constants. Below the threshold, pull PA5 low to switch the heater MOSFET on.
- 2 Capture a 160x120 frame over DCMI.
- 3 Run *RaindropCoverage* on the frame.
- 4 If coverage is at least 5 %, temperature is above 0 C, and 4 s have passed since the last wipe, sweep the servo 60 to 115 degrees and back.
- 5 Print one status line on USART2: temperature, heater state, coverage, the 6-character tile grid, inference microseconds, and whether a wipe ran.

```
if (cam_ok && nn_ok && CameraCaptureRGB888(s_frame))
cover = RaindropCoverage(s_frame, CAM_W, CAM_H, s_grid);
if (cover >= WIPE_THRESHOLD && temp_c > 0.0f && (HAL_GetTick() - last_wipe) > WIPE_COOLDOWN_MS) {
Wipe(); last_wipe = HAL_GetTick();
}
```

The status line is the measurement channel. Logging USART2 during a rain test gives per-frame latency (nn_us) and lets each printed grid be checked against what is visible on the lens, which is how accuracy is scored.

6.4 Build

In STM32CubeIDE: enable DCMI, a DMA2 stream, I2C1 and a 12 to 24 MHz clock output for the camera XCLK in the .ioc; import a tflite-micro tree generated with *create_tflm_tree.py*; compile it as C++17 with TF_LITE_STATIC_MEMORY and CMSIS_NN; add the three firmware files and the generated model array; exclude the old main.c. Link with nano.specs and -u _printf_float so the status line prints floats.

7. Test and verification

- **Subsystem tests.** Servo tested on a servo tester at the machine shop (found a dead motor). Power rails verified at 5 V and 3.3 V. MOSFET output verified with and without a control signal. Temperature readings compared against a reference, giving the 9 C offset.
- **Heuristic.** 100 trials on varied test images.
- **CNN pipeline on the PC.** The from-scratch fallback in *lens_soiling/* has nine pytest unit and integration tests (train, export, infer). The pretrained pipeline writes a JSON report at every stage: port (per-variable match and sample-patch predictions), fine-tune (test accuracy), distillation (student test accuracy), TFLite (int8 vs fp32 agreement, byte size, op list).
- **End-to-end.** Final demo with all subsystems interacting: freezing simulated with a cooler of ice and a 20.5 C threshold, wiper triggered by detected obstruction, LED confirming heater state. Outdoor test of the enclosure.

8. Problems and how they were handled

Problem	Cause	Fix
CNN on the PC lagged the live feed	Per-frame inference in Python on a laptop, no batching	Shipped the OpenCV heuristic for the demo; later moved a distilled int8 CNN onto the MCU
Heuristic false positives on glare and texture	Lens millimetres from the drop surface, so texture is magnified	Edge-based detection with circle fill; then the tile CNN trained on real drops
Pretrained model cannot run on STM32	18 M params, LRN op not in TFLM	Knowledge distillation into a 20 k-param student with only TFLM ops
Second PCB MCU destroyed	Fine-pitch package, hand soldering	Final demo on the Nucleo dev board with the same firmware
Servo not moving	Dead motor, not wiring or code	Confirmed on a servo tester, replaced
No freezing conditions available	Bench test indoors	Cooler of ice with the heater threshold raised to 20.5 C

9. Interview talking points

- **Tradeoff.** Tile classification with a coverage number instead of segmentation: far cheaper on an MCU, and it maps straight onto the wipe rule. You lose drop shape, which you do not need.

- **Tradeoff.** Distillation instead of training the small net from scratch: the teacher's soft labels transfer what it knows about glare, and you get to use published weights instead of collecting a large dataset.
- **Design choice.** Heater on the thermistor, wiper on vision, and a 0 C lockout so the two never conflict.
- **Broke.** The near-lens geometry that made the heuristic false-fire; found with the TA by looking at the mechanical setup, not the code.
- **Next time.** Mount the camera farther from the wet surface. Add a frost class. Use an STM32H7 with more SRAM so a full QVGA frame and a wider student fit.

References: Guo, Akcay, Adey, Breckon, ICIP 2018 (github.com/tobybreckon/raindrop-detection-cnn). Soboleva & Shipitko, RaindropsOnWindshield, 2021 (Zenodo 4680442). Uricar et al., SoilingNet, ITSC 2019. Hinton, Vinyals, Dean, Distilling the Knowledge in a Neural Network, 2015. Howard et al., MobileNets, 2017.